Blockchain consensus protocols

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Arnaldo Tomasino

Gabriele Colapinto

April 16, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Introduction to Blockchain Consensus Protocols **5**

2 Proof-Based Protocols: Verifiable Effort and Resource Commitment **7**
2.1 Proof of Work (PoW) . . . 7
2.1.1 Definition . . . 7
2.1.2 Core mechanism . . . 7
2.1.3 Difficulty Adjustment in Bitcoin Proof of Work . . . 8
2.1.4 Security assumption . . . 10
2.1.5 Drawbacks . . . 10
2.2 Proof of Stake (PoS) . . . 10
2.3 Other Proof-Based Models . . . 11

3 Voting-Based Protocols: Reaching a Quorum **13**
3.1 Practical Byzantine Fault Tolerance (PBFT) . . . 13
3.1.1 Client Request and System Model . . . 13
3.1.2 Multi-Phase Communication in PBFT . . . 13
3.1.3 Execution and Client Reply . . . 16
3.1.4 PBFT in the Context of Blockchains . . . 16
3.1.5 Clarification of Common Misconceptions . . . 17
3.2 Raft . . . 17

1 Introduction to Blockchain Consensus Protocols

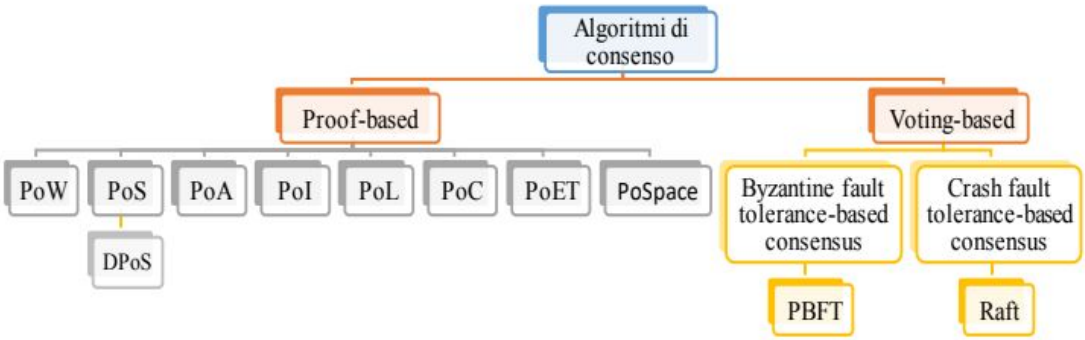

Figure 1.1: Consensus protocols classification

Consensus protocols are the set of rules and procedures that allow a distributed network of independent computers, or nodes, to agree on a single, shared state of the ledger. These mechanisms are the cornerstone of a blockchain's security and integrity, as they ensure that every node maintains an identical copy of the transaction history, even in the presence of network faults or malicious actors attempting to disrupt the system. Without a robust consensus protocol, a decentralized network would be unable to maintain trust among its participants.

Consensus algorithms can be broadly categorized along two primary dimensions (See figure 1.2):

1. **Proof-based vs. Voting-based:** Proof-based algorithms require participants to demonstrate a verifiable commitment of a scarce resource (like computing power or capital) to participate. Voting-based algorithms, by contrast, rely on a set of known participants reaching a formal agreement through a voting process.
2. **Permissionless vs. Permissioned:** This classification refers to the network's access control model, which fundamentally shapes the design of its consensus protocol.

Table 1.1 contrasts these two architectural models.

The most foundational and widely recognized consensus protocol, particularly in permissionless systems, is Proof of Work.

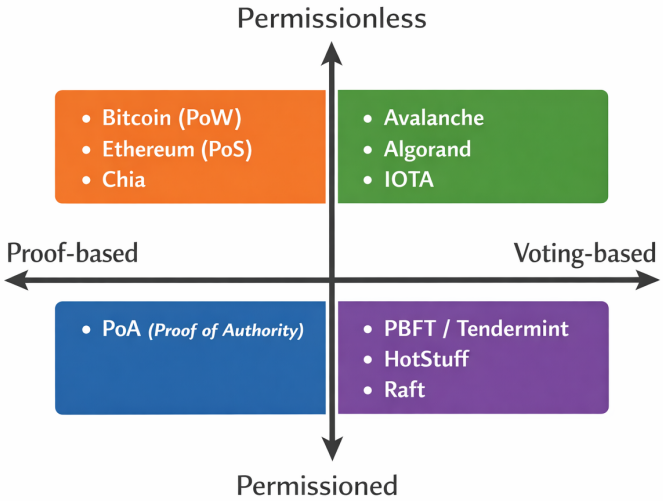

Figure 1.2: Consensus protocol classification graph

Characteristic	Permissionless Blockchain	Permissioned Blockchain
Access Control	Anyone can join the network to read, write, and participate in consensus without prior authorization.	Only authorized parties can join the network and participate in the consensus protocol.
Participant Identity	Participants can be anonymous or pseudonymous. Identity is not a prerequisite for participation.	Participants are known, and their identities must be authenticated to join the network.
Consensus Robustness	Must be highly robust to defend against attacks from unknown and potentially malicious actors (e.g., Sybil attacks).	Can use less computationally intensive protocols, as the set of participants is known and trusted to a degree.
Incentive Model	Participants are typically rewarded with a native cryptocurrency for their contributions to securing the network.	Incentives may not be required, as participants are often organizations with a vested interest in the network's operation.
Example Platforms	Bitcoin, Ethereum	Hyperledger Fabric

Table 1.1: Permissionless vs. Permissioned Blockchains

2 Proof-Based Protocols: Verifiable Effort and Resource Commitment

Proof-based protocols are a class of consensus algorithms that require participants—often called miners or validators—to demonstrate a verifiable commitment of a specific, costly resource. This "proof" serves as a ticket to participate in the consensus process, such as proposing a new block. The resource committed can be computational power, capital (cryptocurrency), disk storage, or even elapsed time. The underlying principle is that by making participation costly, the protocol can deter malicious activity and secure the network.

2.1 Proof of Work (PoW)

2.1.1 Definition

Proof of Work (PoW) is a protocol designed primarily to prevent Sybil attacks, where an attacker creates numerous pseudonymous identities to gain a disproportionate influence over the network. PoW makes the creation of new blocks computationally difficult and therefore expensive, linking block creation rights to real-world economic resources.

2.1.2 Core mechanism

Network participants, known as *miners*, compete to solve a difficult cryptographic challenge. This involves repeatedly hashing the block's header data with different values for a random number called a `nonce` which is part of the block header (See figure 2.1). The goal is to find a `nonce` that results in a `SHA-256` hash that is numerically below a given target value (See figure 2.2). In practice, this is equivalent to finding a hash that starts with a required number of leading zeros.

$$Block\,Hash \leq T\,(target\,value) \Leftrightarrow It\,starts\,with\,n\,zeroes$$

Let us expose how the number of leading zeroes n influences the difficulty. The hashing function `SHA-256` has a fixed output length of 256 bits. Hashing functions are non-invertible meaning that one can't start from a target value an derive the `nonce` that solves the challenge analytically. This means that the nonce is to be guessed by the miners. When a miner tries a random nonce value, they cannot know in advance the result of the hashing function meaning that, to the eyes of the miner, the result of the hashing function is a random sequence of bits. At this point the chances of finding a valid nonce are proportionate to the size of the range of the output value. Until now, we have considered the number of leading zeroes n. But let us look at the result of the hashing function from the other side. The number of bits of the target value (labeled with an X in figure 2.2) is:

$$n_X\,(Target\,value\,bits) = 256\,(Total\,number\,of\,bits) - n\,(Leading\,zeros)$$

The greater the value of n the smaller the range for the target value. For example:

$$\begin{aligned} &For\ n = 10: \quad n_X = 256 - n = 246\ bits \Rightarrow 2^{246}\ possible\ valid\ values \\ &For\ n = 100: \quad n_X = 256 - n = 156\ bits \Rightarrow 2^{156}\ possible\ valid\ values \end{aligned}$$

Let us make a metaphor to better understand the concept: let's say that we are blindfolded and we want to throw a ball in a basket in front of us. Doing so in a small basket is more difficult than doing it in a bigger basket. Increasing n decreases n_X reducing the size of the basket. Conversely, decreasing n increases n_X increasing the size of the basket.

`Increasing` T `decreases` n `and the difficulty of the challenge while decreasing` T `increases` n `and the difficulty of the challenge.`

2.1.3 Difficulty Adjustment in Bitcoin Proof of Work

The Bitcoin protocol is designed to keep the average block generation time approximately constant (about 10 minutes), independently of the total amount of computational power available in the network. This goal is achieved through an automatic *difficulty adjustment* mechanism for the cryptographic challenge.

Why the difficulty must change Satoshi Nakamoto knew that people would have seen bitcoin mining as a profitable activity and expected many people to join the network as miners and the technology to evolve to mine more efficiently. Think about it: miners want the mining task to become simpler in order to employ less resources to get value thus increasing their profit. Considering that both people joining the network and technology advancement increase the number of hashes generated in a given time unit we say that those activities increase the network **hashrate**.

If the difficulty were kept constant:

- an increase in the total network hashrate would cause blocks to be found too quickly;
- a decrease in hashrate would make the blockchain extremely slow.

To prevent these effects, every 2016 blocks the protocol compares the actual time required to generate them with the expected time (2016×10 minutes) and updates the difficulty deterministically:

- if blocks are found too quickly, the difficulty is increased;
- if blocks are found too slowly, the difficulty is decreased.

This mechanism implements a simple distributed feedback control system.

How a single miner perceives difficulty From the perspective of an individual miner, the difficulty does not change the mining procedure itself: the miner continues to compute hashes by varying the nonce and attempting to find a valid one. What changes is the *probability of success per hash*. When the difficulty increases:

- each hash has a lower probability of being valid;
- the expected time to find a block (or a share) increases;
- the expected revenue per unit of time decreases, given a fixed hashrate.

Thus, miners do not perceive difficulty as an operational parameter, but rather as a statistical reduction in the frequency of successful outcomes.

Difficulty, economic incentives, and ASIC investment Difficulty directly influences the economic incentives of mining. When the total network hashrate grows, difficulty increases and mining becomes less profitable per unit of computational power. This creates competitive pressure:

- less efficient miners may be pushed out of the market;
- remaining miners are incentivized to invest in more efficient hardware (ASICs) in order to reduce the cost per hash;
- the system tends toward an equilibrium in which the marginal cost of mining approaches the value of the block reward.

Investment in ASICs is therefore not a direct reaction to difficulty itself, but to the economic competition that difficulty makes explicit.

The lottery ticket metaphor A useful intuitive model for Proof of Work is the lottery metaphor:

- each hash computed by a miner corresponds to buying a lottery ticket;
- a ticket is winning if its number is below the target;
- difficulty determines how rare winning tickets are.

Increasing computational power is equivalent to buying more tickets per second. When too many tickets win too quickly, the protocol makes winning tickets rarer by increasing the difficulty. In this way, regardless of the number of participants or tickets purchased, the frequency of wins remains stable over time.

How does the network change the difficulty of the challenge? Bitcoin adjusts the target every 2016 blocks to keep the block generation rate r approximately equal to 10 minutes. Ideally, $r \cdot 2016 = 10\ min\ \cdot 2016 = 2\ weeks$ so, the difficulty of the challenge is updated every 2 weeks.

To update the difficulty of the challenge we need to know if the old threshold value made the challenge easier or harder than intended. We get this information by considering the actual time elapsed to mine the 2016 block and comparing it with the ideal time of 2 weeks (all in minutes). We call f the ratio between the actual time and the intended time:

$$f = \frac{t_{actual}}{2016 \cdot 10\ min}$$

We use f and the previous target value T_{prev} to calculate the new target value T as follows:

$$T = \begin{cases} \frac{T_{prev}}{4} & f \leq 1/4 \\ f \cdot T_{prev} & 1/4 < f \leq 4 \\ 4 \cdot T_{prev} & f \geq 4 \end{cases}$$

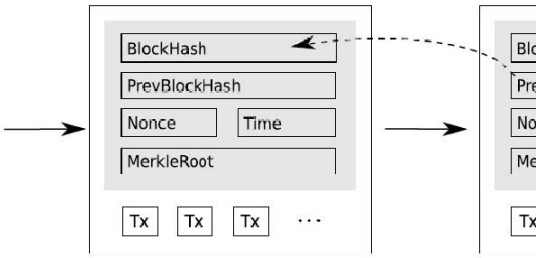

Figure 2.1: Block header

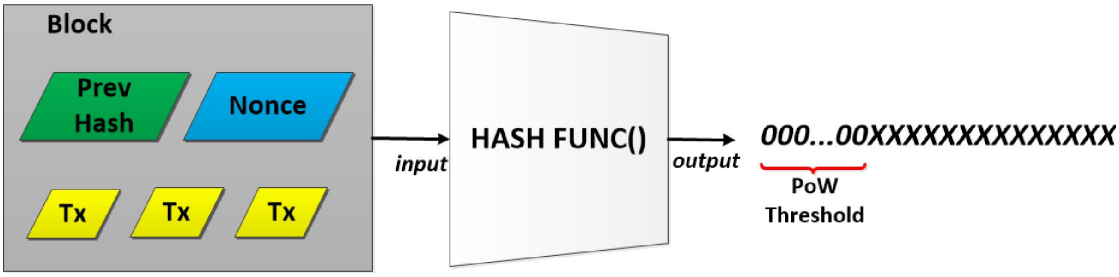

Figure 2.2: PoW hash computation

2.1.4 Security assumption

The fundamental security assumption of PoW is that the blockchain remains secure as long as the majority of the network's computing power is controlled by honest nodes who follow the protocol's rules.

2.1.5 Drawbacks

The most significant criticism of Proof of Work is its extremely high energy consumption, a direct consequence of the immense computational effort required from miners worldwide. The energy consumption problem led many miners to move to China for the lower electricity cost. This created pools of miners which, in turn, generated an additional problem. Mining in pools (basically joining forces) makes mining more efficient but this goes against the decentralization principle which is the foundation of distributed platforms.

Nowadays, 8 pools currently account for 75% of Bitcoin generated blocks. (See figure 2.3)

The presence of mining pools also exposes the network to **eclipse attacks** which are attacks to a specific set of nodes. If a big set of nodes is in the same place, disrupting or isolating a set of nodes becomes simpler.

2.2 Proof of Stake (PoS)

1. **Definition:** Proof of Stake (PoS) has emerged as a leading, energy-efficient alternative to PoW. It replaces the competition based on computational power with one based on capital

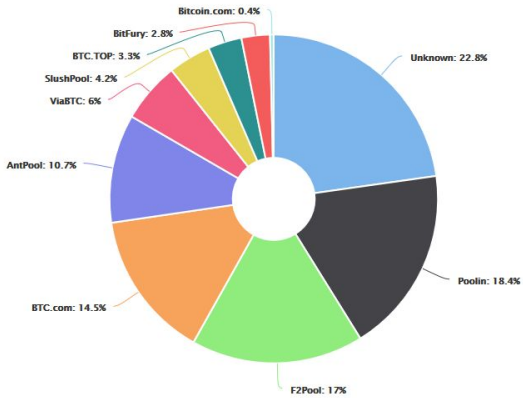

Figure 2.3: Mining pools

commitment.

2. **Core Mechanism:** In PoS systems, participants known as *validators* are selected to create new blocks based on the quantity of the network's native cryptocurrency they are willing to "stake" as collateral. By locking up their own funds, validators are economically incentivized to act honestly, as malicious behavior could result in them losing their staked capital (a penalty known as "slashing").

3. **Key Variants:** Several variations of PoS have been developed to address different design goals and security considerations:
   - **Delegated Proof of Stake (DPoS):** In this model, token holders use their stake to vote for a limited number of delegates (also called "witnesses"). These elected delegates are then responsible for validating transactions and producing blocks on behalf of the network.
   - **Nominated Proof of Stake (NPoS):** This variant involves the election of a committee of validators who then take turns producing blocks in a randomized fashion, sharing the responsibility of maintaining the network.
   - **Liquid Proof of Stake (LPoS):** This model allows token holders to delegate their staking rights to other validators without transferring custody of their tokens. This creates a competitive market for validation services and allows smaller token holders to participate in securing the network.

4. **Vulnerabilities:** Historically, early PoS designs were susceptible to specific vulnerabilities, such as the **"Nothing at Stake"** problem (where validators have no disincentive to support multiple conflicting chains) and **"Long-Range Attacks"** (where an attacker could potentially rewrite a large portion of the blockchain's history). Modern PoS protocols have developed mechanisms to mitigate these risks.

2.3 Other Proof-Based Models

Beyond PoW and PoS, several other novel proof-based consensus models have been proposed, each leveraging a different resource as the basis for network security.

These proof-based systems contrast with protocols that rely not on individual proofs of commitment but on collective agreement among a known group of actors.

3 Voting-Based Protocols: Reaching a Quorum

Voting-based consensus protocols operate on the principle of collective agreement. These mechanisms are primarily employed in permissioned blockchains, where the participants are known and have been granted authorization to join the network. Instead of competing to solve a puzzle, a group of nodes must explicitly vote to reach an agreement, or quorum, on the validity and order of transactions to be added to the ledger.

3.1 Practical Byzantine Fault Tolerance (PBFT)

Practical Byzantine Fault Tolerance (PBFT) is a consensus algorithm designed to achieve *state machine replication* in asynchronous distributed systems in the presence of Byzantine faults. The algorithm guarantees safety and liveness provided that at most f replicas are faulty, assuming a total of $|R| = 3f + 1$ replicas.

PBFT is a *permissioned* and *voting-based* consensus algorithm: the set of participating replicas is fixed and known in advance, and consensus is reached through explicit message exchanges rather than cryptographic proofs such as Proof of Work.

3.1.1 Client Request and System Model

The PBFT protocol operates on *client requests*, which represent application-level operations to be executed by the replicated service. A client request has the form:

$$\langle \text{operation}, \text{timestamp}, \text{client_id} \rangle$$

and is digitally signed by the client to ensure authenticity.

It is important to emphasize that the client request *does not* contain a new version of the blockchain or a freshly mined block. PBFT was originally designed for state machine replication, not for mining-based blockchains. The client merely submits an operation to be ordered and executed consistently by all correct replicas.

The client sends the request to the *primary replica* (leader) of the current view. (See figure 3.1)

3.1.2 Multi-Phase Communication in PBFT

To reach agreement on the execution order of a client request, PBFT uses a three-phase protocol:

1. Pre-prepare
2. Prepare
3. Commit

Each phase contributes to ensuring agreement and preventing equivocation by faulty replicas.

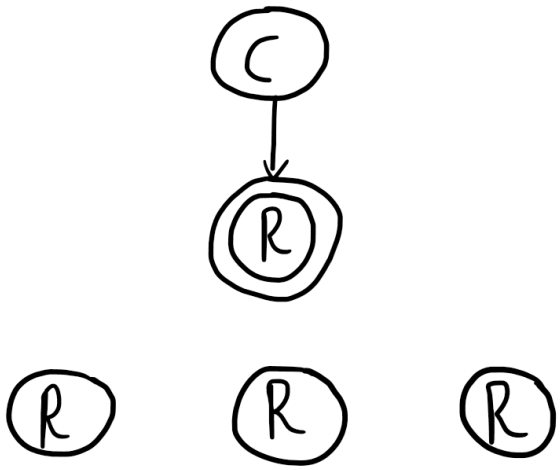

Figure 3.1: PBFT client request

Pre-Prepare Phase

In the pre-prepare phase, the primary replica assigns a sequence number to the client request and broadcasts a `PRE-PREPARE` message to all other replicas (See figure 3.2):

$$\text{PRE-PREPARE} = \langle \text{view}, \text{seq}, \text{digest}(\text{request}) \rangle$$

The message contains:

- the current view number,
- a sequence number defining the total order,
- a cryptographic digest of the client request.

Replicas verify the validity of the message and the correspondence between the digest and the received request. At this stage, the primary is proposing *what* to execute and *in which order*, but no agreement has yet been reached.

Prepare Phase

Upon accepting a pre-prepare message, a replica broadcasts a `PREPARE` message to all other replicas (See figure 3.3):

$$\text{PREPARE} = \langle \text{view}, \text{seq}, \text{digest}(\text{request}), \text{replica_id} \rangle$$

A replica considers a request *prepared* when it has received:

- the corresponding pre-prepare message, and
- $2f$ matching prepare messages from distinct replicas.

This phase ensures that a quorum of replicas agrees on both the content and the ordering of the request, preventing a faulty primary from proposing conflicting values.

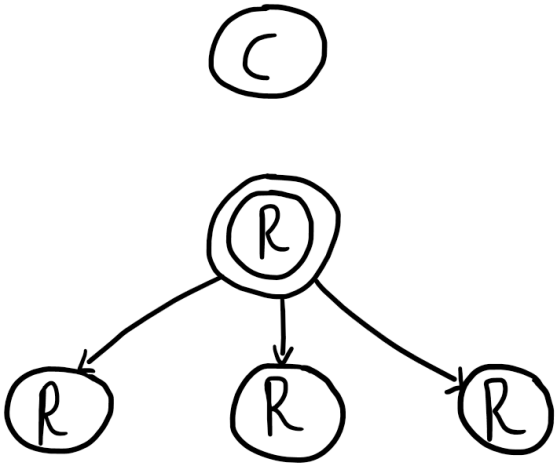

Figure 3.2: PBFT pre-prepare phase

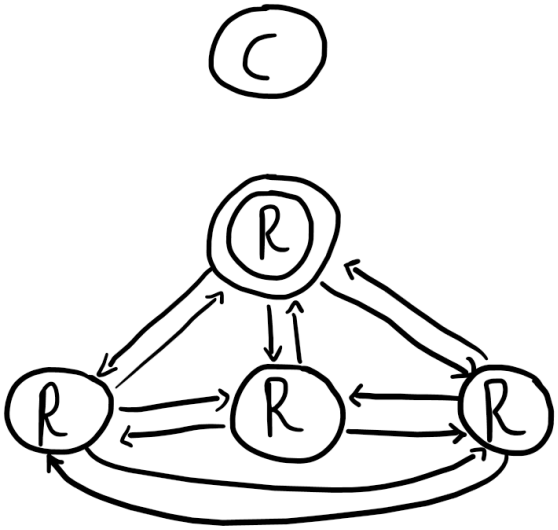

Figure 3.3: PBFT replicas broadcast messages used in the prepare and in the commit phases

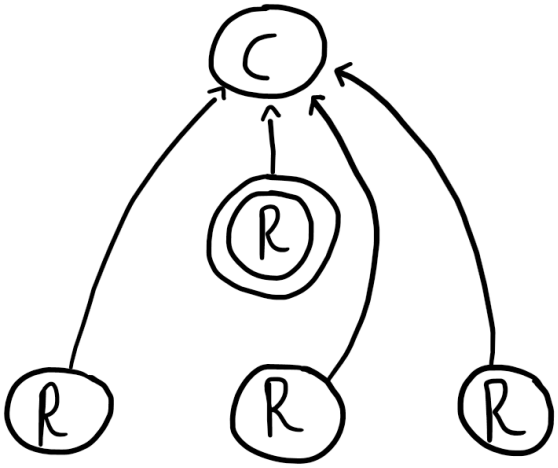

Figure 3.4: PBFT reply messages from the replicas to the client

Commit Phase

Once a request is prepared, replicas broadcast a `COMMIT` message (See figure 3.3):

$$\text{COMMIT} = \langle \text{view}, \text{seq}, \text{digest}(\text{request}), \text{replica_id} \rangle$$

A request is considered *committed* when a replica receives $2f + 1$ matching commit messages. Only after this condition is met does the replica execute the operation. The commit phase guarantees *irreversibility*: once a request is committed, no conflicting request can be decided for the same sequence number.

3.1.3 Execution and Client Reply

After executing the committed operation, each replica sends a `REPLY` message to the client (See figure 3.4). The client waits for $f+1$ matching replies, ensuring that at least one reply originates from a correct replica.

3.1.4 PBFT in the Context of Blockchains

Although PBFT was not originally designed for blockchains, it is widely used as a building block in permissioned blockchain systems such as Tendermint, Hyperledger Fabric, and HotStuff-based protocols.

In a blockchain context:

- the client request typically represents a transaction or smart contract invocation;
- blocks are proposed by designated replicas (proposers), not by clients;
- PBFT phases are used to agree on the validity and ordering of blocks.

Conceptually, the PBFT phases map to blockchain operations as follows:

- **Pre-prepare**: proposal of a candidate block;
- **Prepare**: voting on the proposed block;

- **Commit**: finalization of the block;
- **Execution**: appending the block to the local blockchain.

Therefore, contrary to mining-based blockchains, block creation and finality in PBFT-based systems arise from coordinated voting among known validators rather than from solving cryptographic puzzles.

3.1.5 Clarification of Common Misconceptions

To summarize:

- the client does not submit a block or a blockchain state;
- PBFT orders and executes operations, not mined blocks;
- block structures are an application-layer abstraction built on top of PBFT.

This distinction is essential to correctly understand the role of PBFT in modern permissioned blockchain architectures.

3.2 Raft

1. **Definition:** Raft is a consensus algorithm designed with understandability as a primary goal. Its main purpose is to manage a replicated log in a distributed system, ensuring that all nodes maintain an identical sequence of entries.
2. **Crucial Limitation:** It is essential to note that Raft is **not** Byzantine Fault Tolerant. It operates under the assumption that nodes may fail (e.g., crash or have network issues) but are not malicious. The protocol relies on nodes trusting the elected leader.
3. **Core Mechanism:** The system is leader-based. A single `leader` is elected from among the nodes and is solely responsible for accepting client requests and replicating log entries to the other `follower` nodes. If the leader fails or becomes unresponsive, the follower nodes will time out and initiate a new election to choose a new leader.
4. **Common Use Cases:** Due to its simplicity and fault-tolerant (but not Byzantine-tolerant) nature, Raft is widely used for managing state and configuration in permissioned distributed systems such as the etcd distributed key-value store and the CockroachDB distributed SQL database.

The choice of data structures and a consensus protocol is not an arbitrary design decision. It is a fundamental choice that defines a blockchain's core characteristics, shaping its security model, transaction throughput, degree of decentralization, energy efficiency, and overall governance structure. Understanding these components is therefore essential to evaluating and engineering any blockchain system.